

Rapport de Conception — HospitApp

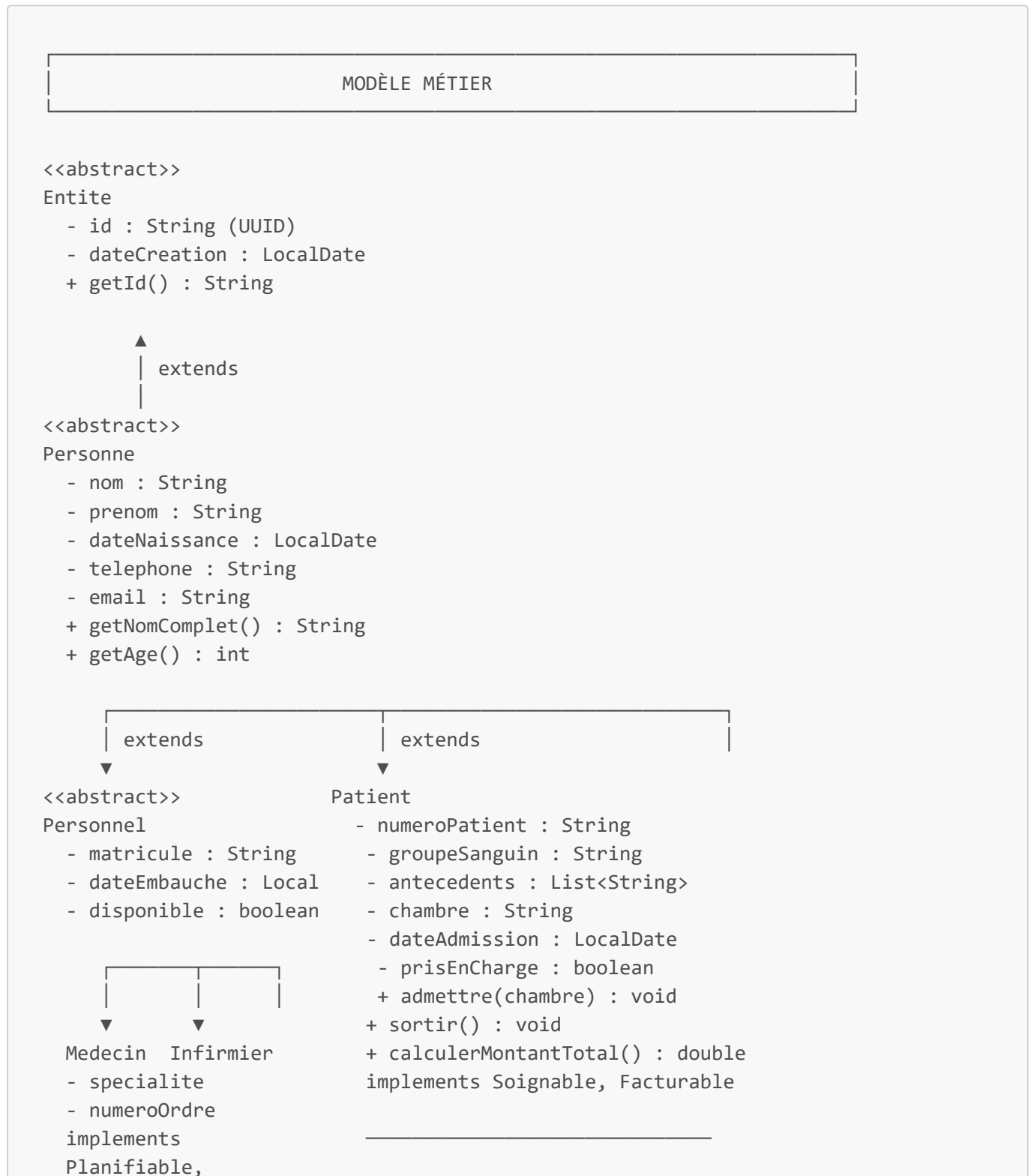
Projet : Gestionnaire Métier avec Interface Graphique (Option B — Servlet/JSP)

Domaine : Gestion hospitalière

Groupe : John DINH, Fasil MOUGAMADOU, Akash ROUBERT

Niveau : Bachelor 3 Informatique — POO Avancée

1. Diagramme de classes UML



```

Soignable                                <<interface>> Soignable
                                         + getSoins() : List<Soin>

                                         <<interface>> Planifiable
                                         + getPlanning() : Map<...>

                                         <<interface>> Facturable
                                         + calculerMontantTotal() : double

                                         <<interface>> Urgence
                                         + getNiveauPriorite() : int
                                         + getLabelPriorite() : String
                                         + getDescriptionUrgence() : String

```

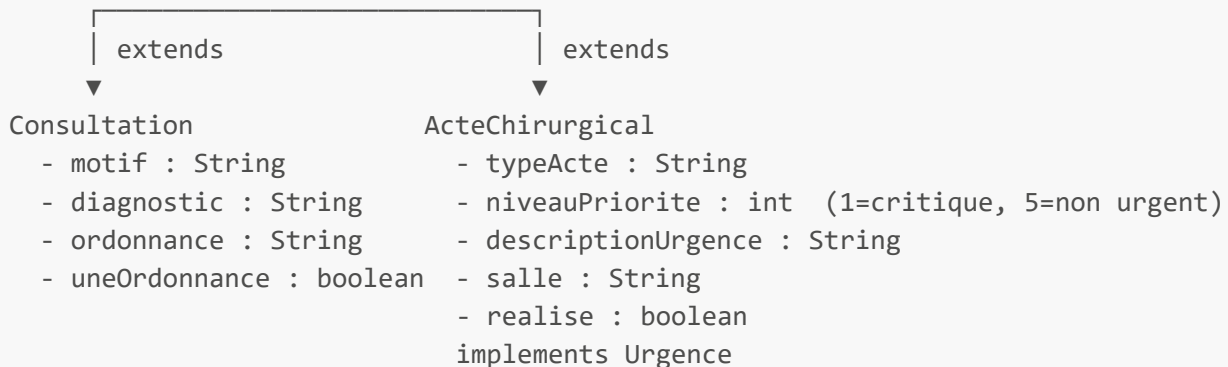
```
<<abstract>>
```

```
Soin
```

```

- id : String (UUID)
- dateSoin : LocalDate
- cout : double
- numeroPatient : String
- matriculeMedecin : String
+ getCout() : double

```



```
<<generic>>
```

```
Registre<T extends Entite>
```

```

- entites : ArrayList<T>
- index : HashMap<String, T>
- ids : HashSet<String>
+ ajouter(T) : void
+ supprimer(id) : boolean
+ trouverParId(id) : T
+ filtrer(Predicate<T>) : List<T>
+ trierPar(Comparator<? super T>) : List<T>
+ compter(Predicate<T>) : long
+ getIndexTrie() : TreeMap<String, T>
+ compterEntites(List<? extends Entite>) : int [static]

```

```
FileUrgences
```

```
- file : PriorityQueue<ActeChirurgical>
```

```
+ ajouterUrgence(ActeChirurgical) : void
+ traiterProchainUrgence() : ActeChirurgical
+ listerUrgencesTriees() : List<ActeChirurgical>
```

COUCHE SERVICE (Singleton)

PatientService	PersonnelService
Registre<Patient>	Registre<Medecin>
+ rechercherEtTrier(...)	Registre<Infirmier>
+ admettre(id, chambre)	+ rechercherMedecins(...)
+ sortir(id)	+ getRepartitionParSpecialite()
SoinService	StatistiqueService
Registre<Consultation>	délègue aux 3 autres services
Registre<ActeChirurgical>	+ getTauxOccupation(capacite)
FileUrgences	+ getAgeMoyenPatientsAdmis()
+ ajouterActeChirurgical()	+ getStatistiquesFinancieresSoins()
+ traiterProchaineUrgence()	

COUCHE WEB (MVC)

```
PatientServlet → PatientService → Registre<Patient>
PersonnelServlet → PersonnelService → Registre<Medecin/Infirmier>
SoinServlet → SoinService → Registre<Consultation/Acte>
UrgenceServlet → SoinService → FileUrgences
StatistiqueServlet → StatistiqueService → (agrège tout)
```

2. Justification des choix de conception orientée objet

Pourquoi **Entite** comme racine abstraite ?

Toute entité du système a besoin d'un identifiant unique et d'une date de création. Plutôt que de répéter ces champs dans **Patient**, **Medecin**, **Soin**... on les centralise dans **Entite**. Cela permet aussi à **Registre<T extends Entite>** de garantir que tout objet stocké possède un **getId()**, sans avoir à faire de cast.

Pourquoi **Personnel** est abstraite et pas une interface ?

Personnel partage du **code** avec ses sous-classes (matricule, disponibilité, date d'embauche, méthodes communes). Une interface ne peut pas partager d'attributs ni d'implémentation. Une classe abstraite est donc le bon choix ici — elle évite la duplication tout en interdisant l'instanciation directe.

Pourquoi des interfaces plutôt qu'un héritage multiple ?

Java n'autorise pas l'héritage multiple de classes. Les interfaces permettent à **Patient** d'être à la fois **Soignable** et **Facturable**, et à **Medecin** d'être **Soignable** et **Planifiable**. Chaque interface représente un **contrat de comportement** indépendant du type concret. C'est le principe de ségrégation des interfaces (ISP).

Pourquoi `Registre<T extends Entite>` et pas `List<Object>` ?

Un type brut (`List` sans paramètre) ou `List<Object>` oblige à faire des casts partout : `(Patient) liste.get(0)`. Le generic borné `<T extends Entite>` garantit la cohérence de type à la compilation. On écrit la logique de CRUD, de filtrage et de tri **une seule fois** pour tous les types. C'est le principe DRY (Don't Repeat Yourself).

Pourquoi `PriorityQueue` pour les urgences ?

La `PriorityQueue` trie automatiquement ses éléments selon un `Comparator` (ici le niveau de priorité). Quand on ajoute un acte critique (niveau 1) après un acte banal (niveau 5), la file se réorganise immédiatement. On n'a pas besoin d'un tri manuel à chaque ajout. C'est exactement ce que fait un service d'urgences réel.

Pourquoi le pattern Singleton pour les services ?

L'application web est multi-requêtes : chaque clic utilisateur génère une nouvelle requête HTTP. Si chaque requête créait sa propre instance de `PatientService`, les données ne seraient pas partagées. Le Singleton garantit qu'il n'existe qu'une seule instance de chaque service, que toutes les requêtes utilisent. Les données en mémoire sont ainsi cohérentes.

Pourquoi deux registres séparés dans `PersonnelService` ?

`Registre<Medecin>` et `Registre<Infirmier>` plutôt qu'un seul `Registre<Personnel>`. Un registre unique obligerait des casts systématiques pour accéder à la spécialité d'un médecin ou au service d'un infirmier. Deux registres distincts évitent les casts et rendent le code plus sûr et plus lisible.

3. Difficultés rencontrées et solutions adoptées

Difficulté 1 — Structure Maven non standard

Problème : Maven attend les sources dans `src/main/java` et les ressources web dans `src/main/webapp`. Notre projet utilise une structure simplifiée (`src/model/`, `src/view/`, etc.) proposée par le sujet.

Solution : configuration du `pom.xml` avec `<sourceDirectory>src</sourceDirectory>` et `<warSourceDirectory>src/view</warSourceDirectory>` dans le plugin Maven WAR. Le build fonctionne correctement.

Difficulté 2 — Tomcat 10 vs Tomcat 9

Problème : nos premières tentatives de déploiement échouaient avec des erreurs `ClassNotFoundException`. La cause : Tomcat 10 utilise `jakarta.servlet.*` et non `javax.servlet.*` (Tomcat 9).

Solution : vérification de la version de Tomcat et mise à jour des dépendances Maven vers `jakarta.servlet-api 6.0.0` et `jakarta.servlet.jsp.jstl 3.0.0`.

Difficulté 3 — Pattern PRG (Post/Redirect/Get)

Problème : après un POST (ajout d'un patient), actualiser la page rechargeait le formulaire et créait un doublon.

Solution : tous les servlets redirigent vers un GET après tout POST avec `response.sendRedirect(...)`. Les messages de succès/erreur sont stockés en session (flash messages) pour survivre à la redirection.

Difficulté 4 — Persistance CSV avec les listes

Problème : certains champs contiennent des listes (antécédents médicaux d'un patient). Le séparateur CSV standard (virgule ou point-virgule) ne peut pas être utilisé à l'intérieur d'un champ sans délimiter.

Solution : les listes sont sérialisées avec le séparateur `|` dans le CSV (ex: "`Diabète|Hypertension`"). À la lecture, on split sur `|`. Ce séparateur n'apparaît jamais dans les données médicales normales.

Difficulté 5 — Tri avec valeurs null

Problème : trier les patients par date d'admission crashe avec `NullPointerException` si un patient n'a jamais été admis (`dateAdmission = null`).

Solution : utilisation de `Comparator.nullsLast(Comparator.naturalOrder())` qui place les nulls en fin de liste dans les deux sens de tri, sans exception.

4. Ce qui aurait été amélioré avec plus de temps

Authentification et gestion des sessions

Actuellement, n'importe qui peut accéder à toutes les pages sans se connecter. Une vraie application hospitalière nécessiterait une authentification (login/mot de passe) avec des rôles (admin, médecin, infirmier) et des restrictions d'accès.

Base de données

La persistance CSV est simple à comprendre mais fragile : pas de transactions, pas de requêtes complexes, risque de corruption en cas d'accès concurrent. Une base de données relationnelle (PostgreSQL ou MySQL) avec JDBC ou JPA aurait été plus robuste.

Pagination

Les tableaux affichent toute la liste en un seul coup. Avec des centaines de patients, la page deviendrait trop longue. Une pagination (page 1, 2, 3...) ou un scroll infini améliorerait l'ergonomie.

Gestion des salles et lits

Le cahier des charges prévoit une gestion des salles et de l'occupation des lits. Nous avons le champ `chambre` dans `Patient`, mais pas de modèle `Salle` dédié avec sa capacité, son type (urgences, chirurgie...) et ses lits numérotés.

Tests automatisés

Il n'y a pas de tests JUnit dans le projet. Des tests unitaires sur les services (vérifier que `rechercherMulticriteres` retourne les bons résultats, que la `PriorityQueue` trie correctement...) auraient sécurisé les refactorisations et facilité la détection de régressions.

Export CSV depuis l'interface

Les données sont sauvegardées en CSV en interne, mais l'utilisateur ne peut pas télécharger un export depuis l'interface. Un bouton "Exporter les patients" générant un fichier CSV téléchargeable aurait été une fonctionnalité bonus utile.